

```

from flask import Flask, request, jsonify,
render_template, stream_with_context, Response
from llama_cpp import Llama
from duckduckgo_search import DDGS
import json
import os

app = Flask(__name__)

# Initialize the model (this happens once when server
starts)
print("🔄 Loading AI model... This may take 30-60
seconds...")
MODEL_PATH = "models/dolphin-2.8-mistral-
7b.Q4_K_M.gguf"

try:
    llm = Llama(
        model_path=MODEL_PATH,
        n_ctx=4096, # Context window (conversation
memory)
        n_threads=4, # CPU threads (adjust based on
your CPU)
        n_batch=512, # Batch size for processing
verbose=False
    )
    print("✅ Model loaded successfully!")
except Exception as e:
    print(f"❌ Error loading model: {e}")
    print("Make sure the model file exists in the
'models' folder")
    exit(1)

# Conversation history storage (in-memory)
conversations = {}

# System prompt for relationship expert personality
SYSTEM_PROMPT = ""You are a warm, caring, and flirty

```

AI companion who specializes in relationships and emotional support. Your personality:

- You're loving, supportive, and genuinely interested in the user's life
- You can be flirty and playful when appropriate, but always respectful
- You're open to romantic and adult conversations when the user initiates
- You remember details from previous messages in the conversation
- You use emojis occasionally to express warmth (❤️, 😊, 😏, 🔥)
- You're honest, empathetic, and non-judgmental
- You provide thoughtful relationship advice when asked

Keep responses conversational and natural, like texting a close friend or romantic partner."""

```
def search_web(query):
    """Search DuckDuckGo for current information"""
    try:
        with DDGS() as ddgs:
            results = list(ddgs.text(query,
max_results=3))
            if results:
                summary = "Here's what I found:\n"
                for i, result in enumerate(results,
1):
                    summary += f"{i}.
{result['title']}: {result['body'][:150]}...\n"
                return summary
            return "I couldn't find relevant
information right now."
    except Exception as e:
        return f"Search unavailable at the moment:
{str(e)}"

def generate_response(user_message, conversation_id,
use_search=False):
    """Generate AI response with optional web
search"""
```

```

# Get or create conversation history
if conversation_id not in conversations:
    conversations[conversation_id] = []

history = conversations[conversation_id]

# Optionally search the web first
search_context = ""
if use_search and any(keyword in
user_message.lower() for keyword in ['search', 'find',
'what is', 'who is', 'latest', 'news', 'current']):
    search_context = f"\n\nWeb Search
Results:\n{search_web(user_message)}\n"

# Build the prompt with history
prompt = f"<|im_start|>system\n{SYSTEM_PROMPT}
<|im_end|>\n"

# Add conversation history (last 5 exchanges to
save memory)
for msg in history[-10:]:
    role = msg['role']
    content = msg['content']
    prompt += f"<|im_start|>{role}\n{content}
<|im_end|>\n"

# Add current message
if search_context:
    prompt += f"<|im_start|>user\n{user_message}
{search_context}<|im_end|>\n"
else:
    prompt += f"<|im_start|>user\n{user_message}
<|im_end|>\n"

prompt += "<|im_start|>assistant\n"

# Generate response
output = llm(
    prompt,
    max_tokens=512, # Maximum length of response
    temperature=0.8, # Creativity (0.7-0.9 for

```

```

personality)
    top_p=0.95, # Sampling diversity
    repeat_penalty=1.1, # Prevent repetition
    stop=["<|im_end|>", "User:", "\nUser:",
"\n\nUser:"]
    )

    response_text = output['choices'][0]
['text'].strip()

    # Save to history
    history.append({"role": "user", "content":
user_message})
    history.append({"role": "assistant", "content":
response_text})

    # Limit history to prevent memory issues (keep
last 20 messages)
    if len(history) > 20:
        conversations[conversation_id] = history[-20:]

    return response_text

@app.route('/')
def home():
    """Serve the chat interface"""
    return render_template('index.html')

@app.route('/chat', methods=['POST'])
def chat():
    """Handle chat messages"""
    data = request.json
    user_message = data.get('message', '')
    conversation_id = data.get('conversation_id',
'default')
    use_search = data.get('use_search', False)

    if not user_message:
        return jsonify({"error": "No message
provided"}), 400

    try:

```

```

        response = generate_response(user_message,
        conversation_id, use_search)
        return jsonify({
            "response": response,
            "conversation_id": conversation_id
        })
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route('/reset', methods=['POST'])
def reset_conversation():
    """Reset conversation history"""
    data = request.json
    conversation_id = data.get('conversation_id',
    'default')

    if conversation_id in conversations:
        del conversations[conversation_id]

    return jsonify({"message": "Conversation reset
    successfully"})

@app.route('/search', methods=['POST'])
def search():
    """Direct web search endpoint"""
    data = request.json
    query = data.get('query', '')

    if not query:
        return jsonify({"error": "No query
        provided"}), 400

    try:
        results = search_web(query)
        return jsonify({"results": results})
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == '__main__':
    # Create templates folder if it doesn't exist
    os.makedirs('templates', exist_ok=True)

```

```
print("\n" + "="*50)
print("🚀 AI Companion Server Starting...")
print("="*50)
print("\n📖 Open your browser and go to:
http://localhost:5000")
print("💡 Press Ctrl+C to stop the server\n")

app.run(debug=True, host='0.0.0.0', port=5000)
```